dia ALGOL manual

An Independent Project on the emulation of the Dartmouth Time-Sharing System - the ALGOL emulator

Antonio Maschio

Independent Project

10 August 2026

ABSTRACT

Introducing the dia ALGOL emulator as was available in the Dartmouth Time-Sharing System arount 1965.

This text is not an ALGOL primer nor an ALGOL manual; I assume you have the most common knowledge about programming, ALGOL and the DTSS; this text is more like a reference document that helps in using correctly the various features, their limits and enhancements.

Even though the bulk of the work is mine, it's undeniable that the Dartmouth documents available at `bitsavers.org` were of immense utility, and constitute the guide of this work.

Nonetheless, coding is my sole responsibility, so any error in the code is due to me and me only. Readers are encouraged to report all errors and inconsistencies (both in the program and in this text) to:

`ing dot antonio dot maschio at gmail dot com`

Support for my efforts is appreciated. Donations can be made via PayPal to

`tbin at libero dot it`

Thanks.

Trasumanar significar per verba
non si poría; però l'essemplo basti
a cui esperïenza grazia serba.
(Dante, Paradiso, Canto I, 1472)

To represent transhumanise in words
Impossible were; the example, then, suffice
Him for whom Grace the experience reserves.
(transl. Henry Wadsworth Longfellow, 1867)

1. A FOREWORD

One of the major arguments about ALGOL 60 is that it is one of the first (if not the first) structured imperative language, the father of Pascal, C, and all the languages derived from them. Structured programming is very important, because it decomposes one problem in smaller sections, that are performed by a single function, or procedure. The main program becomes simpler, getting the data, processing the data with procedures, and printing the results. ALGOL 60 could make all this.

However, one of ALGOL 60's major flaws was that its Standard, called "*Revised Report on the Algorithmic Language ALGOL 60*" (1962, also known as "*Revised Report*" *tout-court*), published as acts of the International Federation for Information Processing (IFIP) conference held in ROme in 1962, solves some but not all the problems of the original 1958 version of the language, and leaves the I/O procedure undefined. On major consequence is that implementers were forced to "invent" their own I/O solutions, making every single version incompatible with the others and making ALGOL 60 programs not portable. See for instance the Dartmouth I/O procedures (as defined in dia), the DEC I/O procedures in the DEC machines of the Seventies, and the ISO I/O defined in the ISO 1538 (1984), both implemented in taxi, my other ALGOL project: three different philosophies, three modes, three different approaches.

In my opinion, I/O solutions are hardly a minor detail[1]: they involve printing to screen, paper, or tape, inputting data from tape, keyboard, or other sources (such as punched cards, as used at Dartmouth in the 1960s). The sole data processing is not possible if data is not given to the program. I/O is probably made by the most frequent commands of any language to enable the communication with the user (the famous `PRINT "HELLO WORLD!"` uses uniquely I/O to define this entry point of any language primer).

Thus, ALGOL 60 was relegated to the role of a language for the initiated - for algorithm scholars - while all "serious" work was destined to be implemented in FORTRAN... Newer versions of ALGOL, such as ALGOL 68 or ALGOL W, had a limited success before the advent of His Majesty PASCAL, after that ALGOL was simply... forgot.

A perfect candidate for my hobby: I decided to revive ALGOL with the project taxi; the same experience gathered in building taxi was reversed in the *deer* project, that since the start had previsions for BASIC, ALGOL and FORTRAN, but that in the first Alpha version was distributed only with the BASIC interpreter dib. ALGOL, with all the modifications imposed by the Dartmouth operators, is a very important part of the project.

[1] Take as example C: the strict language does not include I/O; but the language, since the start, was implemented with libraries, written in C, for I/O, math, type definition an much more. In this regard, no implementer has distributed his C compiler with its own I/O. That's why C has spread worldwide and became one of the most important programming languages ever created!

2. A BIT OF PERSONAL HISTORY AS INTRODUCTION

My relation with ALGOL 60 began after completing the first version of dib, because the project was initially devoted to deer, the Dartmouth DTSS emulator, dib the BASIC interpreter. one ALGOL 60 interpreter, that at that phase didn't even had a name and probably some other pieces, not defined yet (FORTRAN and COBOL being the most probable candidates).

When later I finished decb, the interpreter for the DEC-20 BASIC, in 2011-2012 I thought to realize also the DEC-20 ALGOL, and that it had to be called deca; the project started but never completed.

Much later, in 2019, I decided to start the ALGOL 60 project; this decision involved the abandoning of the DEC side, in favour of a general ISO adherence, with the addition of all the commands in the DEC-20 version because of its diffusion; this project was called taxi and is my main project regarding ALGOL 60.

When in the Summer of 2026 I had to move my site to another location, I resurfaced the whole projects, and seeing that deer needed some maintenance, I happen to think: "*Hey, I've got an ALGOL interpreter now!*". So I copied taxi in the directory of the project, in the meantime baptized dia (is originality not my strong suit?), and started removing, rather than adding... that is removing the ISO statements, the DEC-I/O, the string functions... adding only the particular Dartmouth statements `PRINT`, `DATA` and `READATA` (adding `RESTORE` à-la BASIC) and the Dartmouth line numbering style (atypical in ALGOL), necessary to use the DTSS.

In all, dia may be considered a *reduction* of taxi, but you bet? In conjunction with dib and deer, they are my pride, and probably the project I am most attached to.

3. THE INTERPRETER

In the following are exposed all the characteristic of the ALGOL interpreter. In this text, the Dartmouth ALGOL 60 language is not exposed, if not briefly. You should refer to other texts in the package (and in literature) for news about the ALGOL 60 syntax.

3.1. The available options

When invoked, dia must be invoked with the following syntax:

```
$ dia [options] file
```

Options are:

```
  -b, --root-block  Add external main root block
      --coded       Print error codes, not error messages
      --conditions  Print GPL3 redistribution conditions and exit
  -d, --debug       Debug mode (run-time only)
      --display     Activate -i and -t options
      --errors-list Print a complete errors list and exit
  -h, --help        Display this help and exit
  -i, --id-line     Display program heading
      --purge       Convert program to reserved mode (purging ticks)
  -t, --timing      Display timing after execution
  -v, --version     Display the version banner and exit
      --warranty     Print warranty conditions from GPL3 and exit
```

A version of this brief list appears when dia is invoked as `dia -h`/`dia --help`.

`file` is a textual file with ALGOL code; preferred extension (implicit) is `.alg`.

☞ **Note**: all options that perform a task and then cause dia to exit, without any file elaboration, are generally unaware of the existence of other options specified on the same command line after them; thus, these options should be used alone, and they generally don't need a file name to operate.

Here are all options detailed:

`-b / --root-block`
: Add an implicit outer root block, enclosed by `BEGIN`/`END`; useful if the source has not its own root block. In this case, the `BEGIN` is attached to line 1 (that must be available), and `END` to the last line number + 1. If you happen to have line 1 engaged, you can renumber the file, within deer, invoking `RESEQUENCE`.

`--coded`
: Use error codes instead of the error messages. This architectural choice is not merely a concession to memory constraints, but is grounded in the fact that by decoupling the execution engine from the textual representation of errors, the core interpreter remains completely language-agnostic. The engine's sole responsibility is to detect an anomaly and emit a mathematically precise state token (e.g., `E16`). The translation of this token into human-readable instruction is delegated to the interface layer — in this case, the printed reference card (`ALGERR`) or the manual itself. This separation of concerns ensures that the core computational engine remains compact, fast, and easily localizable without requiring recompilation of the binary.

`-d / --debug`
: activate debug mode; when debug mode is on, the current line is printed before it is executed; this of course alters the output behavior, but you can trace the program flow, and maybe understand why your ALGOL code is not working.

`--display`
: Activate options `-i / --id-line` and `-t / --timing`.

`--errors-list`
: Print a complete list of error codes and messages.

`-h / --help`
: print the standard UNIX help and exit (also `--help`).

`-i / --id-line`
print the program heading; the heading looks like this:

```
PROG24    18:37   JUN. 26, 2026
```

and it's printed before any output of the program. This can be used to document the output. If run within deer this option is not necessary, because deer prints its own heading.

`--purge`
In case the listing you want to execute comes from the IBM Algol, or those derived by it, you may find that commands are enclosed in ticks, like for instance 'INTEGER', to separate the commands from the identifiers set. Using this option to output the program as a purged version; if you intercept it, you can save it to a new purged file; suppose the ticked file is `testd.alg"`; then:

```
$ dia --purge testd > testdp.alg
$ dia testdp
```

From now on you can use the purged file, leaving the original file untouched.

`-t / --timing`
after the program is terminated, print a timer report in the form:

```
TIME:   N SECS.
```

where `N` is the time the execution took. It is in the style of the 1966 Dartmouth BASIC manual (see for instance p. 5) and measure to MSEC to HOUR times. In deer this option is not necessary, because the OS emulator prints its own timing string after the execution of an ALGOL program.
Options `-i` (`--id-line`) and `-t` (`--timing`) can be used together to give the output similar to the one of the DTSS operating system, where the original Dartmouth ALGOL ran. You can activate both using option `--display`.

`-v / --version`
print the standard UNIX version banner, and exit (also `--version`).

To know terms of warranty and conditions, type '`dia --warranty`' and '`dia --conditions`' (these commands print extracts of license, as required by the GPL v3).

3.2. A datum is a datum is a datum...

The ALGOL elements are exposed in the following.

3.2.1. Numbers

Numbers range has the following limits:

- the numerical types are `REAL`, `INTEGER` and `BOOLEAN`.
- the max (positive) real is 1.15792089237E77, identified as `REALMAX` and corresponding to 2^{256}.
- the min (negative) real is -1.15792089237E77, identified as `REALMIN` and corresponding to $-(2^{256})$
- the max (positive) integer is 1073741823, identified as `INTMAX` and corresponding to $2^{30} - 1$
- the min (negative) integer is -1073741824, identified as `INTMIN` and corresponding to $-(2^{30})$
- the smallest real absolute value is 8.63616855551E-78, identified as `EPSILON` and corresponding to 2^{-256}.
- the boolean type can assume only two values: `TRUE` (-1) and `FALSE` (0)

Any number may be input using the ten digits, the prepended minus, the dot and the letter `E` or `$` for powers of ten (`.2` is accepted, `1E2` too, `E2` is not). Follow instructions on the 1966 BASIC manual at page 7: dia is built on the same basics.

3.2.2. Operators

Operators are a delicate question, because they may influence any calculation, and so their management is something to take with care. The algebraic engine turns any expression into a RPN sequence, that is more easily calculated. But this engine uses one-letter operators.

So, for operators with two or more letters I have introduced a substitute. Here they are:

Table 1
Operators substitutions

Operator	Operation	Substitute
+	addition	
-	subtraction	
*	multiplication	
/	division	
^	exponentiation	
\	integer division	
**	exponentiation	^
=	equal test	
<	lesser test	
>	greater test	
=/	not equal test	#
/=	not equal test	#
<>	not equal test	#
<=	lesser or equal test	{
>=	greater or equal test	}
:=	assignment	ASCII 169
AND	bitwise logical and	_
EQUIV	logical equivalence	ASCII 171
IMPLY	logical implication	ASCII 170
MOD	remainder	‘ (inverse tick)
NOT	logical not	~
OR	bitwise logical or	|

The integer division operates a division between its integer operands and return the integer ratio.

3.2.3. Types

The available types for variables, arrays and procedures are:

REAL
INTEGER
BOOLEAN
STRING
DATA (new type introduced by Dartmouth)
LABEL (only for variables and procedures, not for arrays)
INFILE (new type introduced for dia);
OUTFILE (new type introduced for dia);

Contrarily to what established by the 1964 document (and not confirmed in the 2004 version by Kurtz), DATA declaration are global; they are evaluated in the pre-parsing phase, data are collected and stored in proper internal arrays, suitable for the READATA statement. The DATA type, thus, does not follow the other variables scope. The types INFILE and OUTFILE follow the rules of the other types but DATA.

3.2.4. Reserved words

Certain letter combinations are reserved as part of the structure of the language and cannot be used as identifiers or part of identifiers.

For instance, to determine the BEGIN-END levels, an important phase in the pre-processor, taxi looks for each BEGIN-END pair, storing the positions and other data, useful to the run-time phase to know where to jump. But a variable or a procedure named or containing BEGIN or END will influence the pre-processor, which will certainly fail in finding all pairs coupled.

The same can be said for instance for WHILE, FOR, DO and so forth.

Table 2 contains a list of all the reserved words used in dia; identifier should not contain any of the words in this list.

Table 2
Reserved Words

; (semicolon)	IF
ARRAY	INTEGER
BEGIN	LABEL
BOOLEAN	PROCEDURE
COMMENT	REAL
DATA	STRING
DO	SWITCH
ELSE	THEN
END	VALUE
FOR	WHILE

The semicolon is not an identifier of course; it was included in this list because it is substantial, and cannot be employed in purposes other than as a terminator.

A special class of reserved words are the terminator delimiter words (a subset of the previous list, actually), that greatly influence the program structure. In Table 3 they are fully explained.

Table 3
Terminator Words

Terminator Word	Usage
END	Used to terminate a block begun with BEGIN; it may be followed by any free form text, and terminated by a semicolon or dot; the outermost block, called the primary (or root) block, may be terminated by a dot; all the innermost by a semicolon.
ELSE	Used to terminate the THEN part of an IF-THEN-ELSE clause.
;	Semicolon; used to terminate any statement; if the statement is followed by END or ELSE, the semicolon is not mandatory.

All the words in Tables 2 (and in 3) should be used neither for identifiers nor for part of identifiers, or the structure of the program could be greatly ruined. The rest of the Algol words cannot be used to *begin* the identifier, but they may be contained in the name (for instance, AGOTO is legal, GOTOA is not).

All other identifiers (mainly for procedure names) can be used to build identifiers; for instance you can redefine your PRINT() routine, that at all effects will substitute the default one, or you can build your own SIN() procedure.

3.2.5. Variables

Identifiers for variables in ALGOL are more complicated than in BASIC. The rules for the variables name are:

1. All identifiers in a program (included label variables) have to be "declared" before usage. Names may be composed by at most 30 characters, they must start with a letter and be composed by letters and numbers.
2. Identifiers cannot be set equal to or contain any of the reserved words of the language in their name, as in tables 2 and 3 above, while they may contain the names of the non reserved words if the language in their name; besides, variables names should neither start or be set equal to the name of an argument-less procedures. Ignoring these rules can severely interfere with the text interpretation in the pre-processing phase or in the run-time phase.
3. The case of letters, in an identifier name, matters, that is AVOC and avoc are two distinct items (variable or procedure).
4. There is no implied type attached to an identifier (that is, suffixes are meaningless, like $ for strings in advances BASIC).

For example, the following are valid and distinct identifiers:

```
I
i
Alpha
BAND
J4k5
HOUSEHOLDERTRIDIAGONALIZATION
```

Note that `I` and `i` are distinct, and that `BAND` contains the relative reserved word `AND`, but it's a valid identifier name nonetheless. The following are not valid identifiers:

`4P`	it does not begin with a letter
`1`	it is a number
`AND`	it is equal to a literal operator
`BEGIN`	it is equal to a reserved word
`TREND`	it contains a reserved word
`GOTOA`	it begins with an Algol word

As in BASIC, spaces are removed from the source, and this can be used to enhance the creation of the identifier; the following are valid identifiers:

```
VELVET UNDERGROUND,  ONCE AGAIN,  TWO EPSILON
```

which will be seen and used with the names

```
VELVETUNDERGROUND,  ONCEAGAIN,  TWOEPSILON
```

but can be referred to in any place with the spaced declared identifiers.

3.2.6. Arrays

Arrays in dia, as in the original ALGOL, may have one or more dimensions (this system accept at most 16 dimensions); when the dimension is one, the object may be referenced as a vector, when two the object is a matrix (the original 1966 BASIC manual called them respectively lists and tables). The declaration of arrays follows the usual form:

```
10 REAL ARRAY A[0:23],B[1:14,-5:5];
```

As already known, ALGOL accept any interval in the declaration of arrays; any array element is accessed through the same interface:

```
20 Y:=A[2]+B[2,1];
```

using the `[..]` square brackets.

Unlike BASIC, arrays and variables cannot be re-dimensioned; in case two different variables are declared with the same name, the result is, e.g:

```
ERROR: SYMBOL ALREADY DEFINED AT LINE NO. 30.
```

3.2.7. Strings

Generalities

This version of ALGOL 60 may use string variables and string arrays; it's worthwhile noting that the original Dartmouth ALGOL in 1965 could use the string type, so it had no strings (apart from the literal ones) and had no string functions. In planning to add strings to dia, seemed natural adopting also the most common and useful functions, in order to spare the programmers to build their own functions as ALGOL procedures.

These variables and arrays are declared as usual:

```
STRING S;
STRING ARRAY COLLECT[0:10];
```

Strings may be long at most 75 characters.

Byte subscript
Strings sectioning is possible through the byte subscript extraction:

```
STRING S;
S:="MY SOUL AND LIFE";
PRINT("FIFTH LETTER IS","",S.[5]);
```

returns

```
FIFTH LETTER IS  79
```

As you can see, the output is the ASCII value. This technique can also be used to alter the strings.

```
S.[5]:=77;  ! set to ASCII value M in place 5;
PRINT(S);
```

returns

```
MY SMUL AND LIFE
```

String indices start from 1 and the last index points to the last character in the string. The terminator of the string (usually the zero) is not included in the string, and if its index is specified, or even beyond, error code 24 is returned; for instance, asking for the 17th character or beyond, the following is returned:

```
ERROR: SUBSCRIPT OUT OF BOUNDS AT LINE NO. 130.
```

String functions
The function `SIZE(X)` returns the size in characters (bytes) of the string, counting from the first character until the first zero excluded (the zero may be set by the user, using the byte subscript technique, to resize the string)[2].

The `CONCAT(S1,S2,...,SN)` function concatenates two or more strings—separated by commas, provided as literals or contained in string variables.

The `EVAL(S)` function returns the result of the expression contained in `S`, which can be a string literal or a string variable containing the expression itself. If the string is not fully consumed after the result is processed, an error is reported.

String comparison
Two strings can be compared with each other using the usual comparison operators. For example

```
IF S < "ALBA" THEN GO TO L;
```

where `S` is a string variable, string constant or a string procedure. The effect of the comparison is to compare the strings byte-by-byte, the "lesser" string being that with the first lower value byte, working from left to right. Thus `"ABCD"` is less than `"ABCE"` or `"ABCDE"`. Where the strings to be compared are of different byte sizes, then the smaller bytes are regarded as being extended on the right by null bits.

Trailing blanks (spaces or tabs, or any mixture thereof) are treated as equal. Similarly, ASCII strings of different lengths will compare equally if the extra length comprises only spaces and tabs. In all other cases strings of unequal length can only be regarded as equal if the extra length consists entirely of blanks.

Thus, if `S` contains `"  ABCD  "` and `T` contains `"ABCD"` they are treated as equal, and a test such `S=T` would return true.

The cases of letters matter (upper or lower): they will compare equally if cases match. Thus if `S` contains `"A"` and `T`

[2] Please note that in taxi the function `SIZE()` returns the string space (in taxi it's possible changing and redefine the string length); the actual string length (the part not filled with zeroes) in taxi is performed by the function `LEN()`. In dia, being the string not resizable, the function `SIZE()` becomes the function that returns the actual string length, and thus corresponds to `LEN()` in taxi.

contains "a", they are treated as different, and a test such S=T would return false.

String joining

Literal strings may be "prolonged" to the next line, as in the famous example, by not finishing the string with the double quote:

```
20 PRINT("Now is the time for all good men to come
30        to the aid of their party");
```

The string is recomposed in phase of loading, second line number (30 in the example) is consumed, the spaces are all consumed but one, and the text is concatenated to the previous; conditions are:

- The total length of the line cannot be longer than 120 characters
- The total length of the string cannot be longer than 75 characters

3.2.8. Labels

A label is a method of marking a place in a program so that control can be transferred to that point from elsewhere in the program.

dia uses identifiers as labels. These identifiers are placed before statements and are followed by a colon. A label name must begin with a letter and optionally be followed by one or more letters or decimal digits or underscores. A label name, as an identifier, may not contain more than 30 characters.

Numeric labels are permitted, as established in the Revised Report (but were not implemented in the Dartmouth and in the DEC system-10/20 ALGOL).

For example:

```
COMP: X:= X + Y
```

is a statement labelled by COMP.

More than one label can be attached to a statement if required; thus,

```
LAB1:  2000:  Y:= 0 ;
```

NOTE: only digits, letters and the dot are allowed in labels; if you want to use more than one single word for your label, use the dot as a separator. E.g.:

```
SQUARING OPERATIONS:        ! this label is illegal;

SQUARING.OPERATIONS:        ! this label is illegal;

SQUARING_OPERATIONS:        ! valid label;
```

I don’t know what was the behaviour in the Dartmouth ALGOL, but I think anyway that this is a minor issue.

3.2.9. Booleans

Booleans in the Dartmouth ALGOL assume a very special role. Here are listed all the facts about the Boolean type:

- The Boolean Type accepts only the truth values TRUE (-1) and FALSE (0); other integer values cause an error.
- The IF condition must return a Boolean value, in order to evaluate the THEN or ELSE clauses.
- The original Dartmouth ALGOL considered the Boolean type non eligible when using relational operators; for instance A < B worked as long as the two operands are INTEGER or REAL - Boolean types were not admitted. So, two combined relations were forbidden[3].

[3] This rule in some way violates the logic rules. In the paragraph 3.6.3 it explained what I decided for dia...

3.2.10. Operators precedence

Precedence among all the operators is set according to the following schema (higher to lower)

```
level 10: - (unary) NOT
level 9:  ()
level 8:  ^ **
level 7:  * / \ MOD
level 6:  + -
level 5:  < > = <= >= =/
level 4:  AND
level 3:  OR
level 2:  IMPLY
level 1:  EQUIV
```

The operator ^ for powering may be also written as **:

```
A:=3.4^4;
A:=3.4**4;
```

The two expressions above are equivalent.

The operator =/ may be also written as /= and also as <> (the same form of BASIC):

```
IF A /= B THEN ...
IF A =/ B THEN ...
IF A <> B THEN ...
```

The three IF conditions above are equivalent.

3.2.11. Screen

A program line is 75 characters wide and so is screen output; anything beyond the 75th character in program lines will be lost; should they fall beyond the screen limit during output, numbers will be printed on a new line, strings will be cut.

The screen is to be seen as a printed paper flow. Screen columns are numbered 1 to 75, and this is quite different from the terminal (or terminal emulation) you are using, which is at least 80-characters wide. Anyway, this reflects the original input space of the Dartmouth BASIC, so you should never use program lines longer that 75 ***in all*** (line numbers included). Use an editor with characters-count like vim.

I took any care to simulate the output **exactly* as in the original ALGOL, assuming that, since it incorporates the BASIC structures, it behaves just like BASIC. Some differences arise with calculation results, because modern CPUs behave better than ancient ones...

3.2.12. Program lines formatting

A program line is a text line with the 1 to five digits at the start, followed by a single statement with its arguments. You can put any blank space into the line, even in the middle of a statement:

```
10 PRINT ("HELLO, WORLD!");
10 PRINT   ("HELLO, WORLD!"):
10PR INT("HELLO, WORLD!");
10 P R I N      T ("HELLO, WORLD!");
```

Previous lines are all equivalent, since all spaces/tabs are discarded during the line processing (of course, not for spaces/tabs into strings!). The starting line number is a heritage of the DTSS, and served to address errors and sorting lines. Dartmouth line numbers are not part of the language ALGOL and are removed during runtime, retained only for error addressing. As explained in the deer documentation, the sorting of lines is automatic, so they don't need to be sorted.

This said, you can format your listing to make it more readable, like:

```
100 ...
110 FOR I:=1 UNTIL 10 DO BEGIN
120    PRINT (I,"","");
130    FOR J:=1 UNTIL I DO BEGIN
140       PRINT ("-->","",I*J,"");
150    END;
160    PRINT();
170 END
180 ...
```

without any risk of misinterpretation. If you want to use an editor with syntax coloring for ALGOL, or use vim with the syntax file `dia.vim`, - see par. 3.7 - take care to write all statements with consecutive letters, in order to achieve the colored syntax: `PRINT` will be colored, `PR INT` will not (but will work).

Moreover, if you want to run your listings under different ALGOLs, be careful: they may not recognize broken statements.

I don't know if the original Dartmouth ALGOL could accept broken statements like `PR INT` instead of `PRINT`, but I'm sure this is a minor point.

Program lines may also be:

- void lines (or lines composed by spaces, tabs and blanks), which are simply discarded;
- lines with a line number only, which delete (if present) the stored line with that very line number;
- lines beginning with #; they are discarded too, to enable some ALGOL scripting; simply copy the following line as the first line in the ALGOL source file (this is not an original Dartmouth ALGOL feature, of course), whose name, for this example, is `<filename>`:

  ```
  #!/path/to/dia <options>
  ```

  Put all the options you want. Then type from console:

  ```
  $ chmod +x <filename>
  ```

  and from now on you will have a sort of executable in ALGOL.

3.2.13. The rule of the semicolon

As already exposed in Table 3, the semicolon rule in Algol is quite strict but not as strict as the rule in C; the general rule is this:

the semicolon must conclude every command scope, with the exception of those followed by `END` or `ELSE`, and in this case the semicolon is facultative.

In case you want to put text after an `END`, text that will be interpreted as a comment, you have to terminate the text with the proper delimiter - the semicolon - for all not final `END` commands.

The final `END` may be followed by the dot or by nothing, since the Dartmouth ALGOL is not as rigid as the rules of Algol 60 in this regard, but a semicolon after the last `END` will cause an error, because it is not the proper way to terminate a program. The final `END` command may be followed by text as well, that will be interpreted as a comment, but the dot is not required.

3.2.14. Capital letters

All statements must be written in capitals; `PRINT()` and `print()` are not the same, the second raising an error, but dia, differently than the original ALGOL, will gently accept lower letters into strings. The following will behave as expected:

```
10 PRINT("UPPER LETTERS");
20 PRINT("lower letters");
```

There is even a further note, here. No check is done upon lower or upper letters for variables, and the variables memory space of dia is enough for all. So A and a are two distinct variables, A[1,1] and a[1,1] are two distinct arrays elements, PROCEDURE FUNC() and PROCEDURE func() are two distinct functions! See the following example:

```
10 BEGIN
20    INTEGER A,a;
25    A:=24;
30    a:=7;
50    PRINT("Multiplication: ","",A*a);
60 END
```

that has the following output:

```
Multiplication:  168
```

Take this feature with care,

I) because this behavior probably differs from the original ALGOL;

II) because it may make your listing incompatible for other ALGOL interpreters and compilers, for whom A and a identify the same variable.

III) because if used within deer, all input is capitalized, so the previous capabilites are lost.

This said, within dia (used as a standalone interpreter), you can count on twice the variables number of the original ALGOL, twice the number of arrays, twice the number of functions... just in case you need it!

3.2.15. Pre-parsing

As said, dia is an interpreter, not a compiler. Nonetheless, some commands are pre-parsed before execution, and with respect to dib the pre-parsing phase is more robust; the following arguments are considered in the pre-parseing phase:

- All labels are gathered and stored in proper structures during the loading phase. Since no variables are declared yet, the only check is for two equal labels, in which case the interpreter stops. During run-time, each declaring variable is tested to assure that two variables in the same space have not the same name, but also that a variable must not have the name of a stored label[4].
- The final END. statement (with dot, not required) is checked.
- The BEGIN-END sections are mapped; the mapping helps in establishing the end of the section.
- The IF-THEN(-ELSE) sections are mapped; the mapping helps in jumping to the right position in case of the condition evaluated by IF.
- The FOR-DO sections are mapped; the mapping helps in jumping to the right position for the cycle reprise.
- The WHILE-DO sections are mapped; the mapping helps in jumping to the right position for the cycle reprise.
- The PROCEDURE headings are mapped, for the return type, for the start and end of the procedure and for the categorizing the procedures, because they may contain procedures.
- All DATA variables are stored in their proper structures and the variables declared. In virtue of this operation, the DATA line (as in BASIC) can be put anywhere in the listing[5] and during runtime, all DATA statements are ignored.

All errors that have some link with the pre-parsing phase occur even before the first program line is started by Hdia.

[4] This is required by the Dartmouth Algol, though not required by the adopted algorithm in dia. I observe, anyway, that using variables and labels with the same name is not good-practice, and make the listing, even if composed by few lines, quite unreadable. That's why I enabled this control.

[5] This is proved by the program in:
https://timereshared.com/dartmouth-dtss-algol/
where the program TPK is presented with the DATA declaration at the penultimate line. Of course, since the program was compiled, this means only that the DATA statements were processed in full during compilation, so that in runtime the data were already acquired. For an interpreter, like dia, this means that DATA must be processed in the pre-parse phase, with identical results for its positioning in the source.

3.3. The ALGOL language

Here you will find the list of all dia's default statements and functions:

3.3.1. ALGOL 60 standard statements

The semicolon that terminates a command is required only if not followed by `END`, even on a subsequent line. Omitting a semicolon may cause the ignoring of the subsequent instruction, or cause other general errors that should be avoided.

Available statements are

- `ARRAY`: define arrays for dimensions from 1 to 16. If the type is omitted, the `REAL` type is assumed.
- `BEGIN`: start a section, ended with `END`. The final `END` does not require the dot (other ALGOL 60 interpreters may be stricter in this regard).
- `COMMENT`: start a comment, which is ignored in execution; being this a normal comment, it must be followed by a semicolon in order to close the comment, that may extended to several lines[6]. The comment may be started also by the `!` token:

```
COMMENT this is a comment;
! this is a comment as well;
```

- `ELSE`: start the section that is executed in case the `IF` clause is false.
- `END`: close a section started by `BEGIN`. Note that unclosed sections cause a pre-parsing error. The text following `END` and until a semicolon is interpreted as a comment; the same for the text following the last `END` (the final dot is not required but if you put comments, it's suggested). Note that the Dartmouth ALGOL often ended a program writing `END PROGRAM.`, though this was not a requirement of Algol 60.
- `FOR-DO`: execute cycles with a depth of 999 cycles. There are three types of `FOR`:

  1) the `FOR` list, in the form:
     `FOR I := 3,5,10,-24 DO ...`
  2) the classic `FOR-UNTIL`, in the forms:
     `FOR J := B UNTIL N DO ...`
     `FOR J := B STEP I UNTIL N DO ...` if no `STEP` section is provided, the step is assumed to be 1.
  3) the `FOR-WHILE`, in the form:
     `FOR V := E WHILE B DO ...`
     In this case, the `V := E` assignment is done at every cycle, the cycle is executed until the condition `B` is true[7].

  Look at the following example:

```
10 BEGIN
20    INTEGER A; BOOLEAN B;
25    B:=TRUE;
30    FOR A:= 3,5,10,-24 DO PRINT (A,""); PRINT();
40    FOR A:= 1 STEP 2 UNTIL 10 DO PRINT(A,""); PRINT();
45    A:=0;
50    FOR A:= A+1 WHILE B = TRUE DO BEGIN
55       IF A = 5 THEN B:=FALSE; PRINT(A,"");
60    END;
70 END.
```

  The output is:

```
 3             5             10           -24
 1             3             5             7             9
 1             2             3             4             5
```

  As a thumb rule, it's better avoid the sequences `IF ... THEN IF ...` and `IF ... THEN FOR ...`. The restriction dissolves if a compound statement is used, enclosed by `BEGIN` and `END`, like in `IF ... THEN BEGIN IF ... END`

---

[6] Remember: comments must be completed! I say it for direct experience!

[7] This case is useful when the assignment is not fixed, like in the case `FOR I:= I+1 WHILE...`; since `I` is changed during the assignment, the second iteration will use a changed value of `I`, but changing it a second time, and so on. In case of fixed assignment, like as `V := E`, this assures that a datum is read each time; this has sense if `E` is a source (for example a char-by-char reading from a file); in the body of the cycle, the condition may be altered in case the value `V` or some other conditions are found, to terminate the cycle at the last turn.

- `GOTO`: jump to a label. It may also written as `GO TO`.
- `IF..THEN..ELSE..`: executes sections depending on the `IF` condition; in case of true value, the section after `THEN` is executed, otherwise the section following `ELSE` is executed.

  Note that the condition after `IF` is required to be a boolean; so, onlyl the following cases are accepted:

  ```
  IF (relation) THEN ...
  IF B THEN ...
  ```

  provided that `(relation)` is a logical expression made of combinations of `=`, `=/`, `>`, `<`, `>=`, `<=`, with logic connectives `AND` and `OR`, and in this case the expression has a Boolean type, and provided that `B` is a variable of type Boolean (that return a Boolean type); in all other cases, an error is issued.
- `OWN`: execute the `OWN` variables declaration, variables in the body of a procedure that are retained and made available to the calling environment. `OWN` variables and arrays may be of type `INTEGER`, `BOOLEAN`, `REAL`, `LABEL`, `STRING`; if typeless, the `OWN` declaration defaults to `REAL`:
- `STOP`: execute a brutal and immediate stop of the execution; none of the existing structures (`IF-THEN`, `WHILE`, `FOR` and `BEGIN` blocks) are closed. The execution terminates with the message:

  ```
  dia execution interrupted by STOP at line <n>
  ```

  where `<n>` is the (DTSS) line number where the `STOP` occurs.
- `SWITCH` declaration: a switch declaration takes the form of the word `SWITCH` followed by the name of the switch, an assignment (`:=`), and a list of labels, all on the same line (the declaration cannot be broken). These are called switch elements and must be in the scope of the switch declaration.
- `WHILE..DO`: execute the cycle while the condition is true.

Operative notes:

- Line numbers range is 1-99999; as said earlier in this manual, they are not part of the ALGOL languge; in particular, they cannot be used as a destination for `GOTO`.
- A strictly correct syntax is necessary for the parser to work; so (and this is not bad at all) no freedom is left in leaving behind closing parentheses; no doubt: this enhances clarity and eases corrections.
- The powering operator may be followed by unary operators `-` and `+`; e.g.:

  ```
  12^-1E-2
  3.2^+4.2
  A^-E1
  ```

  are all allowed form; in any case, for a better reading, you may enclose the exponent into parenthesis:

  ```
  12^(-1E-2)
  3.2^(+4.2)
  A^(-E1) fam T
  ```

3.3.2. ALGOL 60 Dartmouth enhancements (and more)

The Dartmouth ALGOL creators, being the Algol 60 Revised Report unclear for what about the I/O commands, implemented their own commands to print to screen and to input data, largely following the BASIC statements and outputs. Moreover, when I designed dia, I found that these Dartmouth I/O commands where very good, easy to use and handy, so I provided also a bunch of similar commands to deal with file I/O, something that was beyond conception in the mid Sixties.

The extended features of the Dartmouth ALGOL and the addition made by me are described in the following:

- `INFILE` (dia addition): type for an in-file object; this object may be addressed by `READATA` to read elements from a file. The object is declared as any other object:

  ```
  INFILE FIN;
  ```

At this point, the object FIN is not connected to any file. To connect, simply assign to it the file name (with path, if necessary):

```
FIN := "ODATA";
```

Now, passing this object to READATA will cause the reading of objects from the file and the assigning to the arguments. Objects must be saved in the WRITEDATA protocol (see ahead).

- OUTFILE (dia addition): type for an out-file object; this object may be addressed by WRITEDATA or WRITE to save elements to a file. The object is declared as any other object:

```
OUTFILE FOUT;
```

At this point, the object FOUT is not connected to any file. To connect, simply assign to it the file name (with path, if necessary):

```
FOUT := "OUTDAT"
```

Now, passing this object to WRITE or WRITEDATA will cause the writing of the arguments to file.

- PRINT() (Dartmouth version): display strings or expression built up from numbers, variables and operators; tab and carriage return suppression is possible. To suppress a tab before printing, insert an empty string "" as a parameter before the number; to suppress a carriage return, a "" should be the last parameter in the list. The library that execute the output is taken from the BASIC compiler, so that the output of the two environment is the same.

Note: to emulate the BASIC line:

```
PRINT 2^I;
```

in ALGOL, the following must be used:

```
PRINT(2^I,"","");
```

The first ,"" token symbolizes the semicolon; and since in BASIC the final semicolon instructs BASIC to pack the next printing, we must remove the Carriage Return, so here's why the final token is ,"") is used. To emulate the BASIC LINE:

```
PRINT
```

you can use an empty command:

```
PRINT();
```

that prints a Carriage Return only.

- RANDOMIZE (dia addition): randomize the seed used for the random number generation. This command was not in the original Dartmouth ALGOL.
- READATA() (Dartmouth version with dia additions): read elements in three modes:
  - → from a DATA type object, that must be previously declared with DATA;
  - → from an INFILE object, that must be previously declared with INFILE; the reading occurs from the file, reading objects in the file lines, written accordingly to the WRITEDATA format (see ahead).
  - → if the data object is TELETYPE or KEYBOARD, the reading occurs from keyboard; elements must be input separated by comma, using different separators will cause the error MISSING OPERAND OR DELIMITER;
- RESTORE() (dia addition): reset the element pointer of a specific DATA object[8].

[8] This command was not part of the original Dartmouth ALGOL, but I added it because of its utility and because, since the DATA/READATA mechanism is taken from the BASIC statements DATA/READ, the RESTORE (addressed to the proper DATA structure) seemed a natural consequence.

- `WRITE()` (dia addition): this command acts as `PRINT`, but directs the output to the file, with the same conventions for zones and empty objects that drive the printing. It is suitable for sending program outputs to file, for building documentation or retaining the output data.

  Warning!: the output of `WRITE` is not suitable for reading with `READATA`.
- `WRITEDATA()` (dia addition): this command will print the arguments that are passed to the command, with the following protocol:

  → Numbers are printed with the maximum precision, to let the successive reading not losing decimals.

  → Strings are printed enclosed with double quotes, to let the successive reading recognize strings.

  This command may write to file objects that the command `READATA` can recognize, provided they are are read in the same order as they were written (strings to strings, numbers to numbers).

3.3.3. ALGOL functions

The functions of the Dartmouth ALGOL are:

- `ABS(X)` returns the absolute value of `X`
- `ARCTAN(X)` returns the arc-tangent of `X`, the returned value is in radians
- `COS(X)` returns the cosine of `X` in radians
- `ENTIER(X)` returns the integer part of `X`
- `EXP(X)` returns the exponential of `X`
- `LN(X)` returns the natural logarithm of `X`
- `SIGN(X)` returns the sign of `X`: -1 if negative, 1 if positive, zero if zero
- `SIN(X)` returns the sine of `X` in radians
- `SQRT(X)` returns the square root of `X`

In addition, the Dartmouth ALGOL (and dia) includes the `RANDOM` function, whose value is a pseudo-random number in the range $0 \leq$ `RANDOM` ≤ 1.

Operative notes:

- The trigonometric functions work in radians mode only. Notable is the absence of the function `TAN(X)`, probably because the division `SIN(X)/COS(X)` was always possible and probably because it can be produced immediately through a one-argument `PROCEDURE` that uses calls by value (the simplest).
- The `ENTIER` functions operates like the `INT` BASIC function in version III: as stated by Kurtz in [2], starting with version III, the `INT` function was changed to allow the usage of `INT(X+.5)` both for positive and negative numbers, so that `INT(12.9)` is 12 and `INT(-4.2)` is -5 (this is confirmed by Gottfried with the example #5.2 in [3]). The same is built in dia for `ENTIER`.
- The `RANDOM` function is completely deterministic, and the same values series is returned for every instance of execution (this is exactly the way the Dartmouth ALGOL worked); this helps in configuring a program which needs random number, with a fixed set. This may be rendered more random using `RANDOMIZE`, after which a new seed is set basing on the time of day, so that it's likely the returned values are no more deterministic (rather pseudo-random)[9].

3.3.4. Algol special features

Algol 60 has some notable features that maybe don't belong to any other languages. The following features are not explicit in the Dartmouth ALGOL, but I added them because they are in the Algol 60, and I think that the original writers of the Dartmouth ALGOL could have inserted them hardwired in the code, though not explicitly described.

3.3.4.1. Conditional operands

Algol 60 allows the user to substitute a conditional operand for any operand in an expression by the use of a construction involving

```
IF ..... THEN ..... ELSE.
```

[9] This is different from what established in taxi, where `RANDOM` is always not deterministic, because a `RANDOMIZE` is executed before runtime.

For instance, you can write:

```
I := IF B THEN 1 ELSE 0;
```

This is more compact and of great use in cases such as:

```
J := J + (IF K < 1 THEN 1-K ELSE K-1);
```

Note that the conditional operand must always be bracketed if included in an expression; brackets may be avoided only when it forms the complete expression by itself.

In general, a conditional operand may replace an operand in any arithmetic or Boolean expression. Also, a conditional operand may replace a label and act as an element in a switch list, for example:

```
SWITCH SW := L1, IF B THEN L2 ELSE L3, L4;
```

It is also permitted in an array subscript or in a byte subscript. E.g.:

```
X := A[I, IF L = 0 THEN J ELSE J+1];
```

It must be underlined here that a conditional expression is made of the `THEN` *and* the `ELSE` part, i.e. the `ELSE` part is not optional, because a value must be ensured, either as a result of the `THEN` or as a result of the `ELSE` part.

Besides, the expression `IF..THEN..ELSE..` is interpreted as a value, and must reside on the same line: it cannot be broken. An error is printed if `THEN` or `ELSE` are not found.

3.3.4.2. Conditional expressions and statements

Since a conditional operand may replace any operand in an expression, operands can also be replaced in conditional expressions. Consider the following example:

```
IF (IF B THEN B1 ELSE B2) THEN I := I + 1;
```

The `IF-THEN-ELSE` between brackets may be seen as a number, though the type returned must be Boolean (because in Dartmouth ALGOL the `IF` condition is always a Boolean). It is the same sequence as if it was written as

```
C:=B2;
IF B THEN C:=B1;
IF C THEN I := I + 1;
```

where `C` takes the Boolean value `B1` or `B2` depending on the value of the Boolean `B`.

3.3.4.3. Conditional statements

The reader was introduced to conditional statements of the form

```
IF B THEN S1 ELSE S2
```

in thr previous paragraph. The full power of this type of statement can now be demonstrated. First, `S1` and `S2` can be compound statements or blocks. For example:

```
IF I < 0 THEN
   BEGIN
      I := -I; B:= FALSE
   END
ELSE
   BEGIN
      I := I + 1; GOTO L2
   END;
```

Second, the whole structure of the `IF ..... THEN ..... ELSE` statement can be made more powerful by

using conditional statements within themselves. For example:

```
IF X < 0 THEN X := 0 ELSE IF B THEN GOTO L
```

This is equivalent to the following sequence of statements:

```
    IF NOT X < 0 THEN GOTO L1;
    X := 0; GOTO L2;
L1: IF NOT B THEN GOTO L2;
    GOTO L;
L2:
```

The former method of expression is both briefer and more elegant. Conditional statements take the general form

```
IF B THEN S1 ELSE S2
```

where `S1` and `S2` may both be conditional statements. However, if there is any ambiguity, bracketing using `BEGIN` and `END` must be used to clarify this. Consider the following example:

```
IF B THEN IF X = 0 THEN Y := Z ELSE P := Q;
```

This piece of code can be seen as

```
IF B THEN
   BEGIN
      IF X = 0 THEN Y := Z
   END
ELSE P := Q
```

or

```
IF B THEN
   BEGIN
     IF X = 0 THEN Y := Z ELSE P := Q
   END
```

The first case is interpreted as:

```
    IF NOT B THEN GO TO L1;
    IF NOT X = 0 THEN GO TO L2;
    Y := X: GO TO L2;
L1: P : = Q:
L2:
```

The second case is interpreted as:

```
    IF NOT B THEN GOTO L2;
    IF NOT X = 0 THEN GOTO L1:
    Y := Z; GO TO L2:
L1: P := Q:
L2:
```

ALGOL 60 forbids such ambiguities by forbidding the sequence

```
THEN IF .... THEN .... ELSE;
```

dia, in particular, stops execution if such a condition appears in the listing; if you need an `IF` after `THEN`, enclose the `IF` into a `BEGIN-END` block:

```
IF .... THEN
     BEGIN
      IF ....
     END;
...
```

3.3.4.4. Designational expressions

A designational expression is something that acts as an argument in a GOTO statement, either directly, or indirectly via a formal procedure parameter of type label. This may simply be a label or a switch element. Thus the following are designational expressions:

```
L
IF B THEN L1 ELSE L2
IF X < 0 THEN SW[I] ELSE IF X+Y >= Z THEN TW[J] ELSE L
```

These designational expressions would be used in the following manner:

```
GO TO L;
GOTO IF B THEN L1 ELSE L2;
GOTO IF X < 0 THEN SW[I] ELSE IF X+Y >= Z THEN TW[J] ELSE L;
```

3.3.5. Advanced topics in passing parameters to a procedure

Let's consider some advances techniques for the usage of PROCEDURE.

3.3.5.1. Parameters called by "value"

Calling parameters by *value* is the most common and, except for arrays and strings, the most efficient way to pass a parameter to a procedure. The value of the expression presented in a procedure call, known as the actual parameter, is evaluated on entry to the procedure and assigned to a formal parameter within the procedure. This formal parameter acts like a local variable in the procedure which is initialized, the initial value being that of the actual parameter supplied in the call to the procedure.

In the case of arrays or strings, since a new copy of the array or string is made, this type of parameter-passing for arrays and strings require a (hopefully negligible) execution delay.

Parameters called by value are specified in the VALUE list, because the default passing is by name.

NOTE: labels are always passed by value, independently from the presence of the statement VALUE (which is optional).

3.3.5.2. Parameters called by "name"

Calling parameters by "name" is an useful method of passing a parameter to an ALGOL procedure. Whenever a variable or an array item is passed to a parameter by name, the parameter acts like a substitute of the variable or the array item, that will inherit all the changes applied to the parameter.

WARNING
When passing parameters by name (i.e., omitting the VALUE specification), programmers must not supply dynamic arithmetic expressions that reference a recursive control variable — such as SUM(V, N − 1).

Under standard ALGOL 60 call-by-name semantics, passing an expression like N − 1 causes the interpreter to construct nested evaluation thunks across recursive execution frames. Without explicit environment binding, resolving N at deeper recursion levels creates an unbounded chain of thunk lookups, potentially resulting in stack overflow or infinite evaluation loops.

Design Requirement: Always specify VALUE for scalar recursion counters (e.g., INTEGER N; VALUE N;) or assign the decremented value to a local temporary variable before invoking the procedure recursively.

Look at the following erroneous program:

```
 10 BEGIN REAL T; ARRAY H[1:20]; INTEGER I;
 15    REAL PROCEDURE SUM(V,N);
 20       REAL ARRAY V; INTEGER N;
 25       BEGIN
 40          IF N=1 THEN SUM:=V[1]
 45          ELSE SUM := V[N] + SUM(V, N-1);
 50       END OF SUM;
 55    FOR I := 1 UNTIL 20 DO H[I]:=LN(I)/LN(10);
 65    T:=SUM(H,10);
 70    PRINT("SUM(H,10)=","",T);
 99 END
```

The program above contains two call-by-name errors:

1. At line 65, passing the literal constant `10` as a call-by-name argument is illegal because `N` expects a variable reference.
2. At line 45, passing the expression `N - 1` by name creates an unsupported nested thunk chain during recursion. And trying to reverse the equation in `-1 + N` does not solve either.

To fix both errors, the arguments must be stored in variables prior to the call. At line 65, we can reuse variable `I` and assign `10` to it. At line 45, we can resolve the issue by introducing a new local variable `M` to store the result of the expression (`M := N - 1`), then passing `M` to `SUM`. Here is the corrected program:

```
 10 BEGIN REAL T; ARRAY H[1:20]; INTEGER I;
 15    REAL PROCEDURE SUM(V,N);
 20       REAL ARRAY V; INTEGER N;
 25       BEGIN
 30          INTEGER M;
 35          M:=N-1;
 40          IF N=1 THEN SUM:=V[1]
 45          ELSE SUM := V[N] + SUM(V, M);
 50       END OF SUM;
 55    FOR I := 1 UNTIL 20 DO H[I]:=LN(I)/LN(10);
 60    I:=10;
 65    T:=SUM(H,I);
 70    PRINT("SUM(H,10)=","",T);
 99 END
```

A more idiomatic solution in ALGOL 60, however, is to declare `N` using `VALUE`:

```
 10 BEGIN REAL T; ARRAY H[1:20]; INTEGER I;
 15    REAL PROCEDURE SUM(V,N);
 20       REAL ARRAY V; INTEGER N; VALUE N;
 25       BEGIN
 40          IF N=1 THEN SUM:=V[1]
 45          ELSE SUM := V[N] + SUM(V, N-1);
 50       END OF SUM;
 55    FOR I := 1 UNTIL 20 DO H[I]:=LN(I)/LN(10);
 65    T:=SUM(H,10);
 70    PRINT("SUM(H,10)=","",T);
 99 END
```

The last two cases produce the same output:

```
SUM(H,10)= 6.55976
```

Table 4 shows the different types of formal parameters, with valid actual parameters that can be substituted in a procedure call.

Table 4
Parameters in a Procedure Call

Formal Type	Actual Parameter
Integer Real Boolean	Any arithmetic expression
String	Any string expression
Label	A label (always by VALUE)
Switch	A switch
Integer Array	An array of type integer*
Real Array (or Array)	An array of type real*
Boolean Array	An array of type Boolean*
String Array	An array of type string
Procedure	A non-type procedure
Integer Procedure Real Procedure Boolean Procedure	A procedure of type integer, real, or Boolean
String Procedure	A procedure of type string
Label Procedure	A procedure of type label

(*) In the case where the array parameter is called by value, any arithmetic type (integer or real) array is allowed as an actual parameter. A conversion of the type takes place during the copying process.

You may wonder why there's not the Label array case; this case is performed by the SWITCH declaring procedure, because a switch is a sort of string array with labeling properties. (See SWITCH.)

If instead any *formula* is passed to a reference argument (that is a number, an expression, a function, etc.), the argument is evaluated at run time returning its proper value. This is legal as far as the argument, in the body of the procedure, is used only in the right part of an assignment (that is, in an expression), because the number is legally admitted as an element of an expression.

If instead this argument is used in the left part, that is an assignment to it is tried, an error is raised, because there is no variable associated. Besides, since the calculated value is always referred to the original call, any instantiation would be reset to the value originally passed at each invocation.

See the following example:

```
 10 BEGIN
 20   REAL R;
 30   PROCEDURE TEST(X);
 40   BEGIN
 50     X:=3; PRINT(X);
 60   END;
 70
 80   R:=4;
 90   TEST(R);
100   PRINT(R);
110   TEST(5*4);
120   PRINT(R);
130 END.
```

The first call to the TEST procedure links the reference of R to X, and when X is assigned 3, this value is assigned also to R; the second call to TEST passes a numeric expression, which is inherited by X, but when the assignment is tried, an error is raised. The total output is of course:

```
 3
 3
ERROR: ILLEGAL ASSIGNMENT AT LINE NO. 50.
```

Let's now suppose, absurdly, that the assignment would be possible. The argument X had inherited the value 20 (that is 5*4) in the creation of the procedure instance and the assignment would try to set X=3; the assignment fails in reality, because there is no linked variables, but we have supposed that it succeeds. So, when X is printed, since there was no assignment, it would recall it's former attribution stored in it (remember that is was not passed by value). So that it would print 20 instead of 3. To avoid these inconsistencies dia (as many other compilers/interpreters) bans such assignments, and returns an error message.

Anytime you have to alter a variable in a procedure, you must assure either to create it as a procedure local variable or to use the VALUE specifier while creating the procedure. In this case, the VALUE specifier would simply create a normal variable with the original value passed (20 in the example) that any further assignment would reset to the new assigned value.

Another interesting consequence is this; if you plan to use a string by name in a procedure, you may think that using it in the right part of an assignment as a char subscript is legal and painless; for instance:

```
120  INTEGER I,J;
130  PROCEDURE TEST(X) LETTERS:(L) FIRST:(C);
140  STRING X; INTEGER L,C;
150    BEGIN
160      L:=SIZE(X);
170      C:=X.[1];
180    END;
190  TEST("TODAY IS A GOOD DAY",I,J)
195  PRINT("STRING IS LONG ","",I,"","BYTES, FIRST IS ","",J)
```

The previous piece of code will fail while assigning L:=SIZE(X), because the search of the string is made upon the referenced string, and not in the body of X, but there is no reference string associated, because X was feed with a literal string. The result is the error message of not found identifier:

```
ERROR: ILLEGAL IDENTIFIER AT LINE NO. 160.
```

In these cases, either you associate a string variable S:="TODAY IS A GOOD DAY"; and pass S to TEST or, since X is never instantiated, it can be passed as VALUE. In both cases, you will get the correct answer:

```
STRING IS LONG  19   BYTES, FIRST IS  84
```

3.3.6. Additional methods of commentary

Three further ways of writing commentary are available to the user in addition to COMMENT and ! described in Section 2.4.

Comment After END

Following the delimiter word END, the user may add any commentary, terminated by a semicolon, or the dot in case of the outer block's END; the characters that may appear in such a comment may be any, in any combination (even reserved words if necessary), except for the dot, the terminator words (see Table 2-4), and the comment words ! and COMMENT themselves. For example:

```
   END OF PROC INVERT;
```

I repeat here that the terminator words END, ELSE, the dot ., the semicolon ; and the comment words ! and COMMENT cannot appear in the END commentary: their presence would put the interpreter in confusion (for block and IF structures and for comment inside a comment). So avoid using such characters in such commentary. Use synonyms like 'closure' for END, 'otherwise' for ELSE or 'remark' for COMMENT and use the comma and the colon or the question mark as punctuation signs. Use the dash - as the terminator (dot or semicolon).

Comments Within Procedure Headings

The ISO 1538-1984 specifies that any procedure heading may be enriched with specifiers (from the second argument on), like:

```
SPUR(A) ORDER:(N) RESULT TO:(S);
```

but this technique was illustrated since the late Fifties in specialized reviews, so I assume this was not an unknown feature in the mid Sixties. The feature, enabled in dia, is this. From a closing `)` to the following opening `:(`, any combination of characters (but the semicolon) may be used. The colon is required (also by the DEC system-10/20 ALGOL and the ISO documents) and must be followed by the open parenthesis of the following argument.

Even the call to such procedures may be as such enriched:

```
SPUR(VECTOR) ORDER:(7) RESULT IS:(RES);
```

This call is equivalent to

```
SPUR(VECTOR, 7, RES);
```

Please note that the final closing parenthesis is one (count the couples: they match) and that no space is allowed between one colon and the open parenthesis.

Comments after the last `END`

Any free form text, after the dot closing the outer block ended with last `END` onward, is ignored by the interpreter, so that any free-form text can be put here, program inputs and outputs, info, manual, help and so forth, also using the 'forbidden' characters `END`, `ELSE`, the dot and the semicolon, i.e.:

```
BEGIN      !outer block;
     ...
     ...
     ...
END OF OUTER BLOCK.<-- here begins the further free-form commentary

This program was created by John Smith on Oct. 23, 2016.
and ended on Jan. 13, 2017.
It can be used with the following data:
...
...
```

This free form text is NOT loaded in memory and can be used to store results or any kind of text being physically not part of the listing. For what about the last `END`, the dot is facultative.

3.4. Error messages

The verbose textual error messages of the 1964 Dartmouth ALGOL were adopted by dia with deliberate omissions and some integrations. This architectural choice is not merely a concession to memory constraints, but is grounded in the following principles:

1. The original errors list included some errors that this interpreter simply cannot detect.
2. The original errors list omitted some errors that to me are important (e.g. `DIVISION BY ZERO`).
3. The added error messages were built trying to retain the same spirit of the original messages, so that the difference is negligible, I hope.

On these basis, the errors are the following[10]:

[10] Dartmouth errors 1, 3, 4, 6, 9, 12, 13, 21, 29, 32, 33 and 35 were omitted, in some cases due to choices in letting some specific feature to emerge (for instance the double negation and the case of two consecutive relations), while errors 17 and 23 were unified under one single error message; dia errors `E07`, `E09`, `E10`, `E12`, `E16`, `E17`, `E20`, `E26` and `E39` are new. The correspondence of the Dartmouth codes with the dia error codes are (the Dartmouth codes are into parentheses):

```
 (2) E03 /   (5) E02 /   (7) E13 /   (8) E14 /  (10) E35 /  (11) E18 /  (14) E19
(15) E30 /  (16) E31 /  (17) E01 /  (18) E06 /  (19) E34 /  (20) E29 /  (22) E11
```

```
E01 MISMATCHED PARENTHESES          E24 SUBSCRIPT OUT OF BOUNDS
E02 MISSING OPERAND OR DELIMITER    E25 ERROR IN PROCEDURE CALL
E03 IDENTIFIER TOO LONG             E26 MEMORY OVERFLOW
E04 SYMBOL ALREADY DEFINED          E27 X^Y
E05 ILLEGAL DECLARATION             E28 UNDEFINED LABEL IN PROGRAM
E06 ILLEGAL ARRAY DIMENSION         E29 NON-BOOLEAN EXPRESSION FOLLOWING "IF"
E07 ILLEGAL ASSIGNMENT              E30 ILLEGAL LEFT PART VARIABLE
E08 ILLEGAL STATEMENT               E31 ILLEGAL SUBSCRIPT
E09 ILLEGAL STRING                  E32 UPPER BOUND LESS THAN LOWER BOUND
E10 UNEXPECTED END                  E33 NO COLON IN BOUND PAIR
E11 ILLEGAL IDENTIFIER              E34 SUSPECT MISSING "THEN" OR "ELSE"
E12 DUPLICATED LABEL                E35 ILLEGAL SYMBOL AFTER EXPRESSION
E13 ILLEGAL CONSTANT FORMAT         E36 INTEGER TOO LARGE
E14 EXPONENT OF CONSTANT TOO LARGE  E37 ERROR IN "FOR" STATEMENT
E15 DATA BLOCK NAME MISSING         E38 PROGRAM INCOMPLETE
E16 ILLEGAL DATA BLOCK NAME         E39 ILLEGAL TYPE
E17 OUT OF DATA                     E40 ILLEGAL EXPRESSION IN CALL-BY-NAME
E18 ILLEGAL SYMBOL SEQUENCE         E41 FILE NOT OPEN
E19 ARRAY NOT SUBSCRIPTED           E42 CANNOT OPEN FILE
E20 DIVISION BY ZERO                E43 FILE TYPE MISMATCH
E21 OVERFLOW                        E44 END OF FILE
E22    SQRT    OF    X                      E45    FILE    ALREADY    OPEN

E23 LN OF X                 E46 TOO MUCH FILES OPEN
```

If you are working on a UNIX/Linux terminal, type

```
$ dia --errors-list
```

to see the list of errors codes and messages. If you are working on the DTSS emulator deer, type:

```
EXPLAIN ALGERR
```

to see the list for standard errors (E1-15, E18-E38) and

```
EXPLAIN ALGIO
```

to see the list for the extended I/O (E15-E16, E39-E46).

3.4.1. Standard error messages

The errors messages are here explained:

`E01` `MISMATCHED PARENTHESES`
An open parenthesis is not paired with its relative closing parenthesis, both for `[..]` and `(..)`.

`E02` `MISSING OPERAND OR DELIMITER`
This error quite always identifies the cases where the semicolon is omitted, or something else is found in its place (for instance, a closing parenthesis that is not paired with its relative open parenthesis). You should examine the code closely to identify the error.

`E03` `IDENTIFIER TOO LONG`
A maximum of thirty letters and digits is allowed in one identifier, not counting embedded spaces.

`E04` `SYMBOL ALREADY DEFINED`

---

(23) E39 / (24) E15 / (25) E25 / (26) E08 / (27) E05 / (28) E04 / (30) E33
(31) E32 / (34) E28 / (36) E38 / (37) E37 / (38) E27 / (39) E23 / (40) E22
(41) E24 / (41) E24 / (42) E36 / (43) E21

Please notice that error 26, which original message was `TROUBLE`, has been changed into the more familiar message `ILLEGAL STATEMENT`.

Either a symbol appears in two declarations in the same block, or the name of a variable/array is the same of an existing label.

E05 ILLEGAL DECLARATION
The name of a variable is used before declaration. Specific errors in declarations may be signalled by other messages.

E06 INCORRECT NUMBER OF SUBSCRIPTS
Check the declaration of the array once mode or make sure all forma parameters which are arrays occur with the same number of subscripts each time.

E07 ILLEGAL ASSIGNMENT
In dia, this error appears when a variable cannot be reassigned or when you try to assign a Boolean variable with a non Boolean value.

E08 ILLEGAL STATEMENT
It corresponds to the SYNTAX ERROR of BASIC; this error is printed whenever dia does not understand the syntax of the command, or an nonexistent command is invoked[11].

E09 ILLEGAL STRING
It occurs when a string variable is assigned a non string value or a string function cannot complete.

E10 UNEXPECTED END
This error is met when a procedure, a block or a section terminates unexpectedly (for instance, an END is met without the relative BEGIN).

E11 ILLEGAL IDENTIFIER
This error appears when attempting to use a variable or some other identifier in the wrong place (e.g. as a label)[12].

E12 DUPLICATED LABEL
This error appears when you declare two labels with the same name. Since the labels are gathered before runtime, when this errors appears no line has been executed yet.

E13 ILLEGAL CONSTANT FORMAT
This error appears when you either have two decimal points in one constant or an illegal character following the E or $ signs.

E14 EXPONENT OF CONSTANT TOO LARGE
The maximum exponent allowable is 75 (in normalized form).

E15 DATA BLOCK NAME MISSING
This error appears when DATA cannot find a legal name for the data block;

E16 ILLEGAL DATA BLOCK NAME
This error appears when READATA cannot find any legal DATA object specified as a first argument in its arguments list);

E17 OUT OF DATA
This error appears when READATA cannot find any more data on the DATA object specified in its arguments list);

E18 ILLEGAL SYMBOL SEQUENCE
You have probably left out a variable between to operators, or put one extra operator in the expression.

E19 ARRAY NOT SUBSCRIPTED
A variable declared as an array does not have a subscript expression.

---

[11] It's curious noting that the original message was TROUBLE. Very succinct.

[12] The original text of this message was referred to the label system (it was ILLEGAL LABEL); I have extended it to the whole identifiers system.

`E20 DIVISION BY ZERO`
A division by zero is tried.

`E21 OVERFLOW`
A floating point number in the source is larger than 2ˆ(2ˆ8), or the argument of `EXP` is greater than 176.752531043, facts that exceed the limits of the original machine.

`E22 SQRT OF X`
A negative value is given to the `SQRT` function.

`E23 LN OF X`
A negative or null value is given to the `LN` function.

`E24 SUBSCRIPT OUT OF BOUNDS`
A subscript does not fall within its declared bounds.

`E25 ERROR IN PROCEDURE CALL CALL`
This error appears when a procedure is not callable for some reason (for instance, for a wrong number of arguments).

`E26 MEMORY OVERFLOW`
This error appears when the memory is full in some direction, for instance a `DATA` object has too much objects, too much `FOR` cycles have been found.

`E27 XˆY`
The arguments `X` and `Y` constitute an undefined expression (typically when `X` is negative and `Y` not integer), whose result is undefined.

`E28 UNDEFINED LABEL IN PROGRAM`
The label to which a `GOTO` statement should step does not exist.

`E29 NON-BOOLEAN EXPRESSION FOLLOWING "IF"`
Just that[13].

`30 ILLEGAL LEFT PART VARIABLE`
A constant or an expression occurs to the left of a `:=` sign.

`31 ILLEGAL SUBSCRIPT`
Arrays may not have Boolean subscripts.

`32 UPPER BOUND LESS THAN LOWER BOUND`
You did something like `ARRAY A[10:1]`.

`33 NO COLON IN BOUND PAIR`
You tried to declare an array in a BASIC fashion, like `ARRAY A[10]`.

`34 SUSPECT MISSING "THEN" OR "ELSE"`
It’s the best supposition, but it may indicate a missing closing parenthesis.

`35 ILLEGAL SYMBOL AFTER EXPRESSION`
It’s probably a missing semicolon. Check the code.

`36 INTEGER TOO LARGE`
In computing an integer value, the computer found an integer greater than 2^{30}, which is beyond the power of the Dartmouth ALGOL.

`37 ERROR IN "FOR" STATEMENT`

The original text of this message was `THUNK CALL`.

[13] This is exactly the comment in [1].

There is a messy FOR statement, or some separators is missing. Check the whole statement carefully.

38 PROGRAM INCOMPLETE
The number of END is lesser than the number of BEGIN. Since not enough ENDs are in the program, the blocks are surely wrong defined.

39 ILLEGAL TYPE
In an integer division operation, one or both of the operands is not of type INTEGER.

40 ILLEGAL EXPRESSION IN CALL-BY-NAME
This error signals the necessity to pass variables, neither literal numbers nor expression, to variables set for call-by-name execution in a procedure.

E41 FILE NOT OPEN
Attempted a READATA, WRITE, WRITEDATA, or CLOSE on an INFILE or OUTFILE variable that has not been initialized with a file name assignment.

E42 CANNOT OPEN FILE
The operating system failed to open the specified file name (e.g., file not found on input, or permission denied on output).

E43 FILE TYPE MISMATCH
Attempted an input operation on an OUTFILE variable or an output operation on an INFILE variable.

E44 END OF FILE
A READATA call attempted to read past the end of the input file stream.

E45 FILE ALREADY OPEN
Attempted to reassign a file name to an INFILE or OUTFILE variable without calling CLOSE on it first.

E46 TOO MUCH FILES OPEN
The system exceeded the maximum allowed limit of simultaneous open file channels (STREAMS limit), which is 15 for the present release.

3.4.2. System messages

Sometimes, things go in such way dia cannot manage them. In such cases, errors are printed in lower letters, preceded by the identifier "dia:", and the program inevitably stops. Here's a list of all the system errors:

```
dia: file not found.
dia: memory error.
dia: <> is an unrecognized option. Ignored.
dia: I/O channel error.
```

These error messages are auto-explicative; the <> term matches with a value which is calculated or retrieved during execution, and incorporated into the message. You're likely to never see the memory error message, unless you're OS has a very very tiny memory. The last error should never happen, because there are only two channels in dia, one for input from keyboard, and one for output to screen, but if you use improperly the I/O commands, it may appear.

If you see the memory or I/O messages, please, write to me. You probably used the commands in a way I hadn't anticipated.

3.5. The vim syntax file

In the package you will find the dia.vim file. If you don't use vim, well, skip this paragraph and forget the file. If you use vim as your current editor, here's some instructions for installing the syntax for any dia file (only for UNIX users).

Installing dia.vim
Locate the .vim directory under your $HOME and find the file filetype.vim". Add the two lines

```
" dia (Dartmouth ALGOL)
au BufNewFile,BufRead *.alg     setf dia
```

`(and create it if it doesn't exist); now step into .vim/syntax` and copy here the file `dia.vim`. Done.

Mapping the `dia.vim` syntax

Of course, you may want to use vim on files created with deer, so the extension is not present. In this case, I map the dia syntax within the vim/gvim resource file (for me `$HOME/.gvimrc`), adding the following line:

```
:map <F2> :set syn=dia<CR><Esc>
```

Of course you can map any key you want. From now on, load the file and hit F2: the colored syntax will activate.

Notes on the `dia.vim` file

3.5. Note on the dia.vim file There is something to say about the `dia.vim` file. The minus sign ahead of a number is seen as the negative sign if followed directly by a number; that is -24 is seen as -24 and colored accordingly. But if the number is inside an expression, like:

```
A-24
```

the minus follows the following highlighting rules:

1 it is colored like the following number if there's no blank space between the minus and the first digit, like in A-24

2 it is colored like an operator if there's a blank space between the minus and the first digit, like in A - 24

My advice is to separate the operators into an expression, and keep the minus tight to the number in case this expression or sub-expression (for instance, into parenthesis) begins with a negative number; in any case, note that the minus preceding a variable is always highlighted as an operator.

3.6. The differences with the Dartmouth ALGOL

Well, building an interpreter as emulator of a compiler for a complex language as ALGOL cannot produce a version fully compatible with the original program, mainly because the two productions are based on different scopes, different constraints, different capabilities, different languages and different compilers. So my efforts were directed in building an emulator closer as possible to the original Dartmouth ALGOL, closer but not equal. Here are the main differences:

3.6.1. The superimposed limits

dia, while not necessary by the current engine, has superimposed the following limits, to be adherent to the original Dartmouth ALGOL:

- Dynamic array declarations are not permitted. What is permitted is using variables in array declarations, but these variables must be instantiated and assigned with proper and valid values. Arrays of dimension 1 are not permitted.
- Unsigned integers may not be used as labels. This is directed to BASIC users: Algol uses labels which are identifiers followed by the colon, and are not related to the line numbers that are used in DTSS to input the program. So `GOTO` points to labels, not to line numbers.
- The integer numbers interval is −1073741824. . 1073741823, that is in the range $-(2)^{30} . . 2^{30} - 1$
- The real numbers interval is `8.636169 E-78` and `5.789604 E 76`, that is in the range $[2^{-256} . . 2^{255}]$.
- The smallest real is 8.63616855551E-78 (that is 2^{-256}).
- The statement `IF` must be followed by a Boolean relation, e.g. the case `IF A THEN`, with `A` of integer or real type is not accepted and generates an error.

3.6.2. Limits and different features

The following are the proper differences by the two systems:

- `DATA` declarations are global and not local to the block in which they occur; that is, it does not matter where they are located, they are evaluated in the pre-parsing phase and ignored during Runtime. Moreover, they are available by each `READATA` call, in or out of procedures at any level.

- Numerical constants are not integer or real *per se*; their type is taken into account in a printing process or if they are assigned to a variable; in the source, a constant is simply a constant.
- Arrays elements cannot be used for running variables in the `FOR` statement, which is permitted by the Dartmouth ALGOL and by the Revised Report of the Algol 60;
- For what about the commands recognition, dia, as the original Dartmouth ALGOL, the Algol commands are recognized by the interpreter by the fact that they are both preceded and succeeded by a non-alphanumeric character. Anyway, identifiers may not contain the commands in Tables 2 and 3, because the pre-parsing phase rely on the recognition

3.6.3. Enhancements

The following features represent improvements of dia with respect to the original Dartmouth ALGOL; as such, they allow for easier use, but the user is free to adhere to the original limits of Dartmouth ALGOL if desired.

- There is no limit for programs and array storage; the original Dartmouth ALGOL allowed at most 5000 words (about 2500 bytes); the limits for dia are instead:
  * Maximum number of program lines: 15000
  * Maximum number of variables: 99999
  * Maximum number of an array dimension: 16
  * Maximum length of a program line: 120 characters
  * Maximum length of a string: 75 characters
  * Maximum number of `FOR`: 99999
  * Maximum number of `WHILE`: 99999
  * Maximum number of blocks `BEGIN-END`: 99999
  * Maximum number of blocks `IF-THEN-ELSE`: 99999
  * Maximum number of `PROCEDURE`s: 99999
  * Maximum number of labels: 999
- The limit for the number of identifiers[14] (see above) is 99999; the original Dartmouth ALGOL allowed circa 510 characters, that is about 170 identifiers, if each of which is three characters long or less, or 510 if each of which is one character long: probably sufficient for any student project, but not seriously apt for larger projects.
- The limit for expressions, nesting of parentheses, brackets and other separators in a mathematical expression is established by the stack depth of the RPN solver, that is 1024 elements for each expression;
- There is no limit for digits in a constant. You can type something like: 1.2345678912345678E45 and the system will accept it and store it; Of course not all the digits will be retained...
- There is no limit for constants (literal numbers) in a program; the original Dartmouth ALGOL allowed no more than 64 distinct source program constants;
- The `OWN` feature is enabled;
- The `STRING` type is allowed; the string functions `SIZE`, `CONCAT` and `EVAL` have been added; all these properties were not in the Dartmouth ALGOL;
- The statements `RANDOMIZE` and `RESTORE` have been added, which were not in the Dartmouth ALGOL;
- The function `RANDOM` can be set as non deterministic using `RANDOMIZE`;
- The symbol for numbers in exponential form in output is `E` (e.g. `3.2 E 23`), but in input it may be `E` or `$` (e.g. `3.2E23` or `3.2$23`)[15];
- The operators `AND` and `OR`, if used with boolean arguments (that is 0 or -1), return a Boolean type (i.e. they behave as logical operators), otherwise they behave as bitwise operators and return an integer type[16];

[14] Labels are not saved as variables and constitute a separate archive. As such, they have a separate limit; the original Dartmouth ALGOL used the same memory for all identifiers, so that variables, array and labels were limited as a whole.

[15] This was done because nowadays we are too used to see the letter `E` in front of the exponent, after decades of hand-pocket calculators usage; let's remember that in 1964-1966 pocket calculators were far to be common tools, so at those times there was no accordance on the symbol to be used for the exponential format.

[16] I don't know how these two operators worked in the original Dartmouth ALGOL; in this version of `dia` this seemed a natural consequence of the ALGOL programming.

- The construction `NOT NOT` is logically valid (see any logic text!) and so it's also legal in dia (though it was not in the Dartmouth ALGOL)[17];
- Two relation together are perfectly valid in dia, because their operands may be of type `INTEGER`, `REAL` and `BOOLEAN`; the original Dartmouth ALGOL prevented the calculation of `x=y<z`, because the admissible types for the operands were only `INTEGER` and `REAL`, so that two consecutive relations cause the second relation to have one Boolean argument.
  dia works on a different basis: since the operators in this expression have the same priority, the expression is evaluated as `(x=y)<z`; `x=y` returns a Boolean result (0 or -1); the final expression returns a Boolean result 0 or -1 depending on `z`.
- dia accepts lower characters in strings; the Dartmouth ALGOL could not, I believe;
- dia considers lower-character variables and functions different than corresponding upper-character ones; so `A1` and `a1` are two distinct variables, and `PROCEDURE FNA` and `PROCEDURE fna` are two distinct functions; this feature was not in the original ALGOL;
- The form `GO TO` was not recognized by the Dartmouth ALGOL, but dia lets you use `GOTO` or `GO TO` as you wish. Spaces are removed, before the line analysis, so that both the commands are seen as `GOTO`.
- The blank after the Dartmouth line numbers is required by the Dartmouth ALGOL, but not mandatory for dia; you can type the command directly after the line number, to save on character, if you even need it.
- Assignments are made with implicit conversion between real and integer numbers; if for instance you try to assign a real to an integer type, the real is rounded to the nearest integer and stored. If you try to assign an integer to a real type, the integer is turn to a real without decimals[18].
  Booleans are different: you cannot assign an integer or a real to a Boolean type, because there is no certainty about the fact the value to be assigned is only `TRUE` (-1) or `FALSE` (0); in this case, an error is raised. To work with Booleans, use only, and prefer the predefined constants `TRUE` and `FALSE`.

These differences, as you saw, won't prevent you from writing correct and compatible programs, shareable between dia and a (theoretical) Dartmouth ALGOL mainframe of the period 1964-1966.

Can you hear that? It's the Beatles' "Rubber Soul" playing!

[17] Well, follow me. 1) admitting that `NOT` accepts only Booleans and returns a Boolean type, the `NOT` on the left returns a Boolean (if the argument is a Boolean), and the `NOT` on the right evaluates a Boolean, so the `NOT NOT` has a sense: an error is issued in case the first `NOT` is required to evaluate a non-Boolean; 2) If instead `NOT` accepts only non-Boolean values, and returns only Boolean, the left `NOT` would succeed if a non-Boolean value is given, but then the second `NOT` would fail in any case, or the first `NOT` would fail if required to evaluate a Boolean.
These considerations lead me to design, more simply, that `NOT` accepts `INTEGER`, `REAL` and `BOOLEAN` types and returns a `BOOLEAN` type. So `NOT NOT` is always, anywhere and in any case a legal operation.

[18] For what reported in [1], constants are typified; if they have one dot, they are considered real, if they have no dots, they are integer. In base of such a premise, I expect that the original Dartmouth ALGOL banned the following assignments:

```
INTEGER I; I:=2.4;
 REAL R; R:=10;
 .br Since numbers are numbers, I admit the implicit conversion, to make programmers' life eas-
ier, taking also in consideration what is written at the paragraph 1.3.1 of [7] "When a real expression
is assigned to an integer variable, its value is rounded to the nearest integer".
```

4. CONCLUSIONS

In the end, there are some people I want to thank, because their work made my job easier (or even possible).

The first is professor Heinz Rutishauser (Swiss mathematician and computer scientist), the real inventor of ALGOL and author of the book "*Description of Algol 60*", written in 1967.

The basis of the Algol 60 were taken from the DEC system-10/20 ALGOL programmer's Guide AA-O196C-TK, changing and updating the parts not complying with a mid. Sixties Algol 60. I want to thank all the minds behind this book.

The following I want to thank is Thomas Kurz, author of the document "Algol - The DTSS (GE-235) Version [2], a brief text that served as my guide to modify taxi and turn it into dia.

Last but not least is my wife, **Anna**, who supported my work and put up with my absence while I was holed up in my study trying to write working code; she complained now and then, but she loves me enough to tolerate it...

*****

Finally, I propose this new acronym for ALGOL:

Algebraic **L**ogic **G**eared for **O**riginal **L**earning

I hope you like dia.

5. BIBLIOGRAPHY

The following documents and references played an important role during the design and the coding of dia:

[1] "*To Potential ALGOL Users*", Sept. 29, 1964, a note to Dartmouth ALGOL users by Stephen J. Garland, revised by D. W. Scott March 3, 1965, available as a pdf file at `https://www.bitsavers.org/pdf/dartmouth/dtss/196409_ALGOL.pdf`

[2] "*ALGOL - The DTSS (GE-235) Version*", by Thomas E. Kurtz, 2004 available as a pdf file at `https://www.algol60.org/docs/KurtzAnAlgolOutline.pdf`
A pdf file of chapter XI of the issued volume (which contains the same text) may be obtained through a free subscription at the acm.org portal.

[3] "*Programming with BASIC*", by Byron S. Gottfried, edition 1, 1975 (Italian version, as "*Programmare in BASIC*", Schaum 1982). This book apparently uses version V of BASIC, in a slightly different flavour, because in it executed in Pittsburgh (not Dartmouth) and because it's a DEC-10 machine.

[4] News about John Kemeny may be found at `http://cis-alumni.org/JKemeney.html`

[5] News about Thomas Kurtz may be found at `http://cis-alumni.org/TKurtz.html`

[6] See also `https://www.algol60.org/`

[7] Also interesting and full of themes is the document "Introduction to ALGOL 60 for those who have used other language", by A.N. Habermann (Sept. 1971, Carnegie-Mellon Univ. - Dept. of Computer Science - Pittsburgh, PA), available as a pdf file at https://www.algol60.org/docs/Introduction%20to%20ALGOL%2060%20for%20those%20who%20have%20used%20other%20language.pdf [8] "A Primer of ALGOL 60 Programming", by E.L. Willey, A. d'Agapeyeff, M. Tribe, B.J. Gibbens and M. Clark, 1962, available at https://dn710009.ca.archive.org/0/items/a_primer_of_algol_60_programming/algol60_o.pdf [9] "A Guide to ALGOL Programming", by D. McCraken, 1962, available at https://dn790000.ca.archive.org/0/items/mccracken1962guide/mccracken1962guide.pdf

*****

The draft of this document begun in textual form on June 2026, and the pdf rendition was completed in July 2026.

I hope you like this program and appreciate the work I've done on it. Feedback is anyway greatly expected, and your contributions too. Send any message (greetings, critics, bugs, suggestions, programs and anything you think important) to the address below.

I am Antonio Maschio, and you can reach me at <ing dot antonio dot maschio at gmail dot com>.

Enjoy! It's GPL!

Table of Contents